

ActInf ModelStream 016.1: Eli Sennesh and Tommaso Salvatori "Divide-and-Conquer Predictive Coding"

Source: [YouTube](#) Playlist: ModelStreams Duration: 1:14:02 | Uploaded: Unknown Transcript method: auto_caption

hello and welcome this is active inference model stream 16.1 it's December 6th 2024 we're here with Eli sesh And thomaso salvator discussing divide and conquer predictive coding so thank you both for joining Eli to you for the presentation all right I everyone so um I recall being here once before you know presenting that old paper interception is modeling alast stasis as control and I think at the time I even mentioned that there wasn't really an inference algorithm that like I could just go and apply to all of these kinds of like active inference and predictive coding problems and over roughly the last year let's say you know we've started work this little team of myself howo who is unfortunately not on the stream due to having a 9-to-5 and Tomaso salvator who's here with us you know we've actually been working on this you know like how would we take these theories and you know this predictive coding idea and just turn it into a basian inference algorithm you can use for stuff so you know as background like to really refresh ourselves this is the super short you know neurs version of our talk you know there's like basically theoretical Neuroscience neuro AI or now I don't know Tomaso like what keyword do you guys use at verses I should have asked you maybe or uh we do use Nei actually biop plausible deep learning that's uh a sentence I often use yeah so I think that that dates back from even like before my versus times I guess yeah so the broad theory is that you know predictive coding sort of solves two tasks for the brain and that's credit assignment and beijan inference now like predictive coding is also a thing in machine learning now they do a lot of it at verses Tomaso's done a bunch with it you know but it doesn't always for various reasons we can go into like it sometimes doesn't always scale as well as backprop basically and so we basically started over again I guess you could say with the concept of predictive coding to build our divide and conquer PC algorithm and we basically find that you can use this at roughly the same scale you would with vaes so of course in order to you know test it out we had this um deep latent gaussian model it's actually the same one from I think the Monte

Carlo predictive coding paper out this year we really did this a lot like leaning on you know comparing to other predictive coding papers rather than to some arbitrary Baseline because you know we figured let's if we pitch this as about p as being about biop plausibility then we can sort of maybe actually get results that are good enough to publish any yeah um so you know we racked up our first win there you went other algorithms that were related or inspired us train really but actually interesting is you know basically we would say all right you have yeah oh um you know after you know demonstrating that this quote unquote wins like okay it works well enough the which oh this entire slide yes okay yeah so our dcpc in addition to being a predictive coding algorithm is a particle algorithm so it's training a particle Cloud so to speak to approximate the posterior distribution in aian inference problem so as such we compared it to this algorithm particle gradient descent pgd from which we were drawing pretty direct inspiration and then we also compared it to you know lengen predictive coding out at icml this year and so we compared on you know the freshed Inception distance that people use in deep learning rather than on you know the um variational bound to the surprise you know we didn't compare on log evidence because that's you know you can often get very bad visual samples from a model that has a very good log evidence actually so in order to you know sort of say like okay that you know log evidence is not the only thing that counts we you know compared on this fresh Inception distance over here we've got some samples from our pseudo posterior predictive distribution so it's a proper predictive distribution but constructed from a pseudo posterior pseudo in the sense that like we're fitting I think a small gaussian mixture model to our big particle cloud of the actual posterior distribution and then having done that we say all right now we'll draw samples from this GMM as a PSE suda posterior you know then pass those latents through the generative model again to generate samples and you know the actual neural architecture here is basically a one level vae like you just chop off the encoder from a one level vae that would be used in these Baseline papers use the same neural architecture in P torch and then basically just got you know slightly better results visually by just improving the inference algorithm and the training okay did did you manage not to lose that cool okay so sort of the interesting stuff about it you know especially compared to previous work is how this actually works as I said it's a particle algorithm so we're actually drawing samples I think I said before we went live you know this is sort of the year of Monte Carlo predictive coding methods we one of maybe you know three or four that are out this year so after you've like initialized your graphical model by just doing you know ancestor sampling to get yourself some initial you know Cloud what you can do during you know every

inference pass is start from the bottom up so that's you know the right over here would be the quote unquote bottom of the hierarchy you can calculate you know your prediction errors which are the same kind of thing as in previous predictive coding work you know there score functions with respect to the latent variable and the thing that we actually do differently from other work is that rather than just taking like a score function as a local gradient of the of let's say rather than taking a score function as like a direct gradient of the variational free energy we take score functions with respect to the log complete conditional distribution so like a coordinate update distribution if you have you know this variable Z_1 then it's complete conditional is the distribution of this variable Z_1 given its entire markof blanket so given both you know the predictions coming in from above it and the observations coming in from below so that's why rather than one you know error unit we he we have an error unit that connects you know Z_1 to the observation below it we also have an error unit that sort of recurrently tries to draw Z_1 back towards its local prior until that prior is changed by an update at Z_2 so if you think about you know how ban inference works you know ban inference is just what you get when you have one distribution and it has to be normalized but you're trying to fit it as closely as you can to both the likelihood and the prior at the same time and so when you try to do that locally within a big structured graphical model you know the logical posterior to go after is the complete conditional and that's what we actually did and I want to shout out how here because he's not on the stream but his previous work on advertised population Gibbs Samplers is really what showed that this kind of sampling from the complete conditionals could work at scale and you know like inspired this project pretty directly like this is kind of follow-up work to you know his paper on that and the way that we you know sort of make this work in terms of constructing a free energy or in terms of making sure we target the correct joint distribution is that we can use Lan proposals you know if you have a prediction error from $e_{\text{sub } Z_0}$ and another you know $e_{\text{down sub one}}$ add them together you have the gradient of the log complete conditional now it's the log unnormalized complete conditional but that's fine because you know the gradient doesn't like you know when you take a gradient of a log density the normalizing constant falls out and so you can Target that with you know a Anin Dynamics proposal which just says take your previous prediction you know multiply your prediction error by a learning rate add the prediction error to the previous prediction and then add some gaussian noise you know you have to balance like the you have to balance the magnitude of the gaussian noise with the learning rate in a certain way but like that's a detail really it's saying let's be in the latent space then let's follow the gradient and then let's inject a

little bit of noise so we explore most of the time or rather sorry so we explore some of the time while exploiting whenever the gradient signal is strong enough and then you know I note that we use sequential Monte Carlo here and what that's actually for is that sequential Monte Carlo algorithms give you a whole set of rules for how you can start from you know the prop $\mathcal{A}$ that you sample from then go to the local Target that you are actually trying to you know that you're trying to sample from and then combine those in you know a neat way to get samples from the Complete Joint distribution so you would do this you know start here from the data on the right do this predictive coding step for Z_1 be done with it repeat the same predictive coding step for Z_2 but now with a new prediction error from Z_1 down here repeat all the way up here and then you have a bunch of samples from different complete conditionals and you have to come up with some way to say well what's my free energy or an SMC terms what's my importance weight and that's really where you know that's really where like the SMC technique becomes helpful is for proving the whole thing is correct so if you think you know we named it divide and conquer because you know what we're essentially doing is we divide the graphical model into these complete conditional units then we conquer each of them with you know basically a Monte Carlo predictive coding proposal and then conquer the whole thing by recombining them so it really is like a divide and conquer algorithm in the classic sense also the reviewers wanted us to name it something clearer than the old name all right so then you know that's a summary of the talk really and then we can go into like details of the paper and you know anything else we want to talk about but broadly the idea here is that you know predictive coding as a theory imp you know enjoys biological plausibility for approximate beian inference which is you know actually this really hard task like you know in deep learning world people use all of these awkward neural network arrangements to try and amortize inference and then the people who are real purists are just doing Markov chain Monte Carlo because amortized inference isn't good enough for them and to be fair it's often not very good if you don't you know structure your amortized posterior like just so and more to the point in Neuroscience you know like predictive coding theories have often been you know built on like mean field assumptions about the approximate posterior so then you just have a coordinate wise representation of the approximate posterior and it's pretty well known that like this can't actually get you a very close approximation to a correlated structured posterior so that was where the trouble was that was the problem that we took a first step on solving you know and we got it to work well enough that you can substitute it for neural networks in some of these cases where people use neural networks and we're going to be at the

poster session next Friday at nups if anyone wants to come and talk to us you know and of course we have to mention the places that we work and our affiliations all right awesome thank you um thank you I'll stop the screen share yeah I'll I'll ask some questions and if anyone in the live chat has questions they can write um maybe just starting with the structured part what do you mean that it's a structured problem Oh yes so like this is the experim you know originally I looked at this and said well how do we actually replicate how results from advertised population Gibbs and that was like a fullon Time series model in which there were you know like some local variables that were updated at each time step but also some Global random variables that were standing you know at the hierarchical level above so you know that was really like I thought okay we need to tackle these kinds of structured problems because you know if you I mean go look at I'm trying to think like you know honestly most of the time most interesting generative models are going to be kind of structured if there's more than one variable and it's not just IID sampling so any kind of hierarchical Baye you know time series structure dynamical system structure like all of this stuff that you find in cognitive science and Neuroscience so so commonly and yet which is often you know we've often sort of given up on doing anything better than particle filtering or amortized inference if you look in the machine learning World okay and the graphical structure you showed was a linear hierarchical model or it was going from side to side but that's just graphical convention so do you ever need to account for branching or any other kinds of structures or how do you divide and conquer when there's also splitting and merging so I mentioned about Markov blankets right as long as you're in a directed A cyl graphical model then there's still a notion of a marov blanket for every single random variable that's the thing that you divide you know the graphical model into is variables with their markup blankets and you know this is sort of intended to be a bit free energy principle perhaps bringing the notebook up would help um I'll ask the two questions so return to them as as you see fit the first is how do you specify the model what does it look like before training and what does get updated during training and then the second question is how do you take that step as a model between the structure of a given problem like the handwriting digits and adapting that to what is that something more generic or does that graph need to already from some other source have structure that's amenable to the [Music] Target so the target density that we would try to sample from is defined by the graphical model structure and like that yeah that has to come from from somewhere so conventionally you know if you're doing machine learning engineering then that's sort of up to your choice as the user of our algorithm you know it depends on what like

domain knowledge you have about your application and if we were talking about brains then that structure of the generative model is something that evolved you know and there could you know and I should mention you know the recent verses work like of course you can also have you know structure learning problems you know throughout like development but you know throughout development and through certain behavioral tasks but like that gets you a bit beyond the realm of just graphical models and so you know we we basically aimed at just a given graphical model take a graphical model as given and try to do inference in it you can work your way up to the harder stuff later in practice we we often use deep neural networks because we we try to replicate the experiments of other papers so we had like multi-layers and the convolutional models yeah maybe share the notebook Andor the repo working up yeah yeah oh the GitHub repo is like yeah we can share that pretty easily yeah going to put this first in the zoom chat then share my screen yeah so in the GitHub repo we try we attempt just zoom in a little bit that's actually lack sure yeah yeah so in the GitHub repo minus a requirements file we try to share enough that you could Hypothetically download and run this like just from your command line you would need you know a python environment that's got the necessary stuff installed but all of that is standard stuff so if you wanted to just rerun the mnist experiment you would literally run this command on your command line and then you could also you know start your tensor board and watch it train you know then we include like evaluation notebooks that you know basically would help you to to replicate the figures from the paper and Yeah we actually have yeah we also share you know some pre-trained like weights and particles I know that's probably a bit of a colliding name with like weights and biases in this case you know the weights include the bias es weights would mean neural network weights and then particles would mean you know the Frozen samples from the posterior distribution for random variables and then of course we just give you know the paper or sorry the table of results and as a small bonus to everyone else who has to try and replicate our results we include an implementation of the descript ized gaussian distribution which is otherwise like buried in the diffusion models literature thank you very much yeah and it's you know under MIT licens so you know you can pretty much go ahead and like Fork this contribute that it's in the Universal probabilistic programming language pyro since that's basically like the largest uh deep probabilistic programming language let's take a look at some of the notebooks uh let's see that one's going to look like garbage this one's going to look like not garbage yeah so what this does is you know it's basically like Pi torch and Pi torch lightning you know so you're going to like read some Json file that's going to give you you

know your data your model on your trainer then you know you're going to remind the trainer to just load the weights and particles from a checkpoint you know the random variables are saved as part of the PyTorch Lightning checkpoints yeah it loads them blah blah blah blah right there when it says 357,000 trainable parameters what is that number coming from that is the number of neural network weights like that is the number of float 64 in the neural network weights so that's actually a relatively small number because this was sort of a toy problem and normally it would be more in the millions and where where were those combinatorics specified uh right here in the architecture specification that's after it though or yeah because you have to load the thing to print the thing okay okay that just printing it out but you specified it elsewhere what the structure would actually be loaded it in printed it out okay yeah the um the Json file you know tells you like which modules in the python code that you have to load and all of that so if we go look at you know what is the actual training file that we were loading here it tells you you know load this model module pass it these hyper params checkpoint this often you know load this data set with like this hyperparameter in batch size you know here's like learning rates and the number of particles you know the number of sweeps meaning like passes over the graphical model you know that you do per training step yeah this is like um relatively standard deep learning type stuff yeah and when we've loaded the architecture we actually print it out you know so it's a simple like isotropic gaussian prior and then we're running it through you know a linear layer an activation function you know a deconvolution layer some batch you know the you know each layer includes batch normalization you know two more convolutional layers with their batch normalization and nonlinearity two more convolutional layers again and then finally a deconvolutional layer that gives you the correct image size and channel number well that's rude anyway yeah so then we have this method we call load particles which just tells you that's really for loading the particles into GPU memory once you know which data items you're testing on again like relatively standard you know deep stuff you know then here we say we're going to open a python context Handler by you know clamping the graphical model to the pre-trained particles that we loaded from disk so you know if you load your posterior particle cloud from disk then you have to say to graphical model I'm testing right now don't do any additional inference steps just use these samples you know as your distribution so then we run we average over particles to get our reconstructions of you know training images now we're going to plot them here's the actual reconstructions so like they're you know kind of blur like you expect from a vae but it shows that you know things did actually work then you say okay I don't want to look

at reconstructions of training images you know I want to know do I have a trained generative model so you clear all that stuff out and you're going to get all of the training and validation you know particles from dis for training your you know for fitting your pseudo posterior and you're going to predict some new images you know using the posterior then lo and behold you know you plot out the samples from the pseudo posterior they're kind of blurry and they're not as varied as you would sort of like them to be but that is what happens when you have a very simple generative model like you know if you go back to this neural architecture here this is nothing like a state-of-the-art you know diffusion model or whatever that you would use for image generation it's really just a bunch of neural layers stuck together mapping a latent variable to an observation yeah and then we're you know having generated those we go through we calculate you know some metrics across the validation data under a series of different random seeds we can also sample from the prior rather than the pseudo posterior and that gives us lower visual quality but more variation more realistic variation and then you know we calculate the statistics that we reported in the paper plus some statistics we don't report in the paper now that I think of it like the um effective sample size that one's a good one for inference nerds you know you divide that by the number of particles you're using and it gives you a sort of percentage that tells you you know how well are you actually fitting the true posterior distribution with this batch of samples and in our case the answer is yeah about 92% effective samples thanks maybe could we walk through a similar Logic for the Mist handwriting yeah totally uh let's see but I think it would be almost equivalent it's just that we use a different model um yeah it's pretty much the same but with you'll notice there's quote unquote three decoders here so each of these is actually performing you know a random gussian sampling step in between so this is a proper hierarchical base lan problem so you have a prior then you have a forward conditional and a forward conditional so you have three levels of latent variables and then finally you have your likelihood what leads you to use those here like do you sweep or scan over 2 3 four layers do you explore out features equals other multipliers or what what led you to this specification for the lightning dcpc so in this case we took this exact specification from Monty Carlo predictive coding which is over here so like we you know we wanted to compare inference algorithms head-to-head so we put you know picked exactly the same generative model structure and you know replicated it awesome yeah that makes a lot of sense and it helps with the cont uity and and isolating the inference algorithmic component is there a in a methods or procedural step anywhere other than these or or are all that are needed these Json configs and the notebooks like this so this is runnable like

this should be runnable if you actually just download it and you know if you download it and follow the links to download you know the pre-trained weights put them in the path that the notebook expects here then you should just be able to run this it should be point and click I say should because you know like I've tried it but I've tried it once cool definitely opportunities for people to explore um and then what about the training yes step so that would be done from the command line as the repo readme tells us that's this step and you watch on tensor board and make a manual call based on diagnostics or how else do you determine I mean I would generally say let it run at least for these EXP experiments let it run to completion like you know I know that some if I'm stopping something partway in because of the Diagnostics I see on tensor flow you know like for instance a very noisy staticky like loss curve well that's how I know when it's working you know the experiments that are mentioned in the paper are actually you know known to work like you can just leave that running until it's done and you will get you know subject to differences in your random number generator more or less the result we got in the paper I actually want to say like this is one of the nice things about the more like sort of the culture of machine learning these days is that everything is really expected to be not just replicable but like idiot proof replicable as in if you cannot just run this by rote and get the same result then reviewers don't believe the result is real which is you know very much for like replicability of science the way it actually should be so we looked at faces and digits let's say somebody were looking at another kind of image classification problem cat cats and dogs so what what would need to be adapted what what kind of data would be uh in in for the training step and then to what extent do these same lightning dcpc specs structures work or to what extent did the first or the secondary layers need to be adapted to Alternative cases MH so this thing that gets called a graph inside the lightning dcpc you'd have to write a new one of classes for a new problem so let's say for instance that you want to do semi-supervised you know beijan learning my old adviser from my PhD had a paper about that you know so if you're doing say cats or dogs then you're going to want to specify you know a new graphical structure that's rooted in a prior over your latent embedding and then it should have you know a pair of likelihoods essentially you know one being to the class label which is going to be it's semisupervised so let's say you only get class labels on about half samples and the other is going to be you know for the actual image which is going to be fully observed you know and then after writing what is it a um a high torch lightning data module for your data set which like tells you you know when which sorry it handles figuring out which data points are semi are supervised versus unsupervised because it's a semi-supervised problem and then yeah you

would put that together and run it more. Like you know you would write another little JSON file and just run it and hopefully you would get hopefully if your um you know architecture for the generative model is rich enough then your latent embedding space should begin to capture you know differences between the supervised classes and be you know and give you good prediction on classification and then if you wanted to do classification style testing or evaluation you know you would present a whole new image as the observation and then just perform joint Bas and inference on both the latent embedding and the um class label and then check that your class labels are good so to speak you know for an unobserved class label you're not getting a one hot Vector you're getting you know a series of logits or like an element on the Simplex that specifies probabilities of class membership so hence I saying good rather than absolutely correct you never get a completely absolute yes no answer out of a classifier at test time at least I think in general you could also generalize it to different kind of data of data types so it does not have to be for example continuous data as images it could be you could generate discrete distribution or you can also generate time series distribution this is something that that has not been tested in this paper but in theory the theoretical framework would allow to do so yeah like we have an appendix in the paper that extends the math to discrete distributions you know that does take a mathematical extension and then that you know would take some more coding so like we don't you know make any promises there so in in past dreams and and Journeys Eli as you mentioned we discussed with Dean and others the anticipatory allostasis and then tomasa we discussed causal modeling so where where would you triangulate this work or how how do you see it as sort of drawing on or complimenting or or not but it's just interesting that it's it's not a homeostatic model often in the organismal ethological active inference there's a lot of focus on organismal behavior in these kinds of topics homeostasis allostasis being critical there and then in the basian modeling space often there's a focus on interpretability of smaller models and carving nature at the joints and all of that not this more machine learning sort of specify it in a sequence of convolutional layers so how do you see all just just having worked on those different areas is how to each of you see I mean because clearly this is a relevant contribution in advance so in in what direction or how do you relate it with those prior pieces I think it depends in which direction you want to you want to push this work a little bit forward so for example if we look at the machine learning side and and we want to scale it up to larger models larger convolutional networks you you don't get interpretability because because of this method I think the same interpretability you would have with the with the old algorithm you have them you have them with this one

what you what you gain apparently what this work shows is the is that you get better performance but then of course if you apply if you apply to for example uh directed graphical models or Basia networks that have a specific hous of structure then you can observe in that case you have the you're able to to stay you can do for example the the intervention kind uh queries that I that I presented here like a year ago or something like that and uh but I think I think that's not in the algorithm is more in the kind of model you you pick up yeah is the kind of task that you want to solve and whether you care about it or not so let's say maybe for generating faces for now we did not care about having an interpretable mod interpretable model what would you say answer would um I would sort of compare it to something like a car engine you know you need to design a car engine of a certain size and Power in order to put in a certain you know in order to put in a certain like weight class of vehicle but that doesn't actually determine you know whether that vehicle is a Honda or a Jaguar if it's the same size and you know weight class then you can actually use a very similar engine inside both of them and I think that's sort of on the one hand the both the Beauty and the pain in the ass of working with beijan methods is that you know the promise on the sticker is that here's ban methods and you're going to solve a very broad range of problems with by reducing them to the same problem which is Ban inference and that's you know sort of what the free energy principle is all about is saying it's all really just reducible to variational inference right but now what you do find if you go look at like pmdp or a lot of you know free energy type papers is that they'll design a model for the paper in order to model some piece of interesting organismic Behavior but they'll include some real restrictions on the expressivity of that model which is that it has to be solvable by something like say variational message passing so then there's the question of well how do you stop restricting the model class according to what inference algorithm you're going to use and that was really like the dirty secr of basion methods that I learned in grad school was you're always restricting the model class to what you have a good inference algorithm for or at absolutely Last Resort you know running something like plain old Metropolis Hastings with a gou and random walk proposal and just waiting for like three weeks of compute hoping so you know what we actually want to do here is to say we have a better engine like literally we have a better inference engine you don't have to you know now structure an entire neural network architecture of um inference proposals you know you don't have to play graduate student descent with your inference algorithm too much you know let's assuming that you know brains have some kind of wondrous generic inference algorithm that solves so many different basan problems all at once you know let's try to model that and then

just use it because it'll be more powerful and more efficient than you know customizing to the particular generative model you want and you know really it's like first baby steps but it is encouraging that you know if you I think we show in the table yes like we trained a variational auto encoder you know as a baseline to say well what if you're doing variational inference by actually you know training a neural network to do it to do it for you you and it's encouraging to show that we can do a little bit better with like a generic inference algorithm that doesn't need a separate architecture you know compared to like build a second neural network and train it jointly with the first one I think something that is also exciting about this I mean at least for me is that we like we've been we've been working a lot with Prive coding algorithm for machine learning in and I guess you can clearly separate them in like supervised learning and unsupervised learning and in supervised learning for example you train like large convolutional models on c for 10 C for 100 or Tiny image net it seems that we've kind of like hit a momentary wall so it's up to a certain point it's becoming really hard to scale up those algorithms and and I think like the old community and it's it's not only Prive coding like when I talk with for example equilibrium propagation pre people with the with all the like biop plausible deep Learning Community there's a there's a kind of similar problem that is a when the model is more than seven eight layers deep it doesn't perform well and for example scaling it up to image net is a pain so a lot of people are taking a step back and say okay let's do a little bit more research to find out why and generalize and and try to improve the performance in when using for example predictive coding for generative AI so to to generate images to sample uh data points in an unsupervised way the first problem that we had is that like in machine learning we had a couple of until like two years ago it was a mostly a deterministic algorithm so we were taking mostly R and Bard's version which was the classification one there was no stochasticity so a bunch of people including me started publishing like associative memory papers it's like I give it a data set and I can uh by giving it a little bit of information you can regenerate exactly the same data points and then in the last year and a half as I said uh some people started developing sampling algorithms and they've been they and basically the the papers before ours a little bit either they do it on on mist and fashion mist or they do it on like colored images like CIFAR 10 CIFAR 100 but they use some kind of gradient descent so it's not a completely local message passing algorithm and and what we did in our paper is that we have a local message passing algorithm and we compare against theirs by using basically exactly the same architectures exactly the same number of airx I think in some cases even exactly the same learning rate and things like that as a small correction we aren't actually a message

passing algorithm yeah like we're not technically any form of some product message passing no this a local algorithm let's say yes local algorithm yes and uh and and we do well like for example the images of faces you've seen may not be may not seem perfect but they are at least from my perspective better than expected and the thing is we haven't I don't I don't feel like we have reached a wall in terms of performance like we've tested only on the architectures that other people have used before and we do well and for now there's little knowledge on how much this can this can scale up maybe you scale it up like two layers and doesn't work anymore that that's something that we we don't know yet but potentially would be interesting to for example something I was messaging with Eli earlier this week is can we scale it up to large larger decoder only Transformer models for example not like of course llms or things like that but for example something that is a like a four five six uh layers de Transformer model that would already be an uh like a quite an interesting result definitely not an easy one but I think there is there is definitely room to improve what's the the results of this paper could you also maybe highlight how how do you see local update rules as superet subset what have you of message passing what differentiates the local locality of connectivity from something that you would explicitly Eli call a message as pasted Ah that's an interesting question I guess this a I think for message passing you you could call like all the belief propagation algorithm or at least basically some kind this class of algorithms where you clearly state which information has to be passed from from basically one latent variable to the other while uh local message passing and and they and those kind of algorithm they're all local so is a since since you define them to be this way they they they must have local information well well probably if you generalize to local message passing in generals there are some proposals maybe including this one in which maybe things are not supposed to be local like maybe you can like since it's a structured learning algorithm you may in theory get information from other from lighten variables that are more far away but is but then when you actually do the computation they are local but that's that's an answer I'm giving now to the spot maybe has a better thought on this so I would say that message passing algorithms differ from ours in both their means and their goals and really it's the goals actually that they're starting from so that family of algorithms you know when when people were working on it maybe I don't know 20 years ago what they were really trying to do was to get a summary of a posterior marginal distribution so they just wanted to be able to pick out one latent variable for an inference query or a decision they needed to make say what is the posterior on this variable given the data not given any settings for all the other latent variables and that sort of restricts you know the space of tasks that you

can really address with message passing and it also you know does end up meaning that you know what you're basically doing is some form of some product or really integral product you know algorithm and that gets you all the way out to I think probabilistic circuits are the most current and advanced form of some product based like exact inference algorithm and they only support tree-shaped you know generative models rather than those that are you know directed a cyclic graphs in general so trees rather than forests and we said well let's try targeting you know the full joint distribution of all the latent variables together partly because that's the most general form of evasion inference problem and partially because you know if you have something like a hierarchical Time series model from the active inference literature you might need several levels of the hierarchy you might need the Latent variables across those levels in order to make you know an action decision you know several levels of hierarchy might actually parameterize your policy or the thing that defines the inference problem you care about solving you know as a modeler that's very interesting again to the sort of settings that a lot of active inference literature has tackled in these cases it's like digit in pixel intensities Etc digit classification generativity of digits out how how would it be similar or different with something like digit pixels in tas's Behavior out or robot arm out are you leveraging this uh Identity or or at least uh structural congruence with the recognition in the generativity or like a vae can you have something different coming in and have the output be something that's categorically different than the type that was used as input like action so I think one of the advantages of defining anything as a basian influence problem is that it really changes your conception of what's an input versus an output variable you know for a vasion inference problem you start with a graphical model you clamp several variables to be observed data those observed data are now your inputs you know your output is any aspect of the posterior distribution that you care to examine so we don't have to be outputting you know classification logits we could also be outputting you know yeah we could be outputting you know functions you know you could have say an active inference time series module where your agent sort of gets some data at every step of computation performs a full sweep over the entire you know history of latent variables and then and only then um you know makes a decision about what action it's going to take which really highlights The Importance of Being able to specify the full joint distribution and then clamp down with Nuance on what you want to be taking as observable but kind of clamping down from a full joint and then just specifying and then having an inference algorithm that will facilitate that locally oper operationally rather than trying to amidst the constraints of which inference procedures you can perform work within

subclasses of graphical models that abide by constraint it's almost like it's a far cry in some ways from 2022 textbook figure 4.3 the kind of pdps and the matrix multiplication that it gets implemented in pmdp the sorts of pedagogical cases that do highlight certain key moves this though looks and feels so much more like a machine learning work and and um it it's um really interesting though what this Ena you also shape the work with respect to the venue you want to submit to I was just going to say now that I found when we were discussing which experiments to to run the first question is are we going to submit a new RS yes and it's going to be okay then it's going to be mostly image generation and the yeah and also also a little bit the venue and a little bit what did prior work do they IM generation we picked out image generation because uncon you know predictive coding networks had not yet been used to get you know this quality of result on unconditional image generation so what other than the conference next week what ways might you be taking it I think maybe then the I mean probably there are many future directions I I guess from the from the perspective interests me the most I would be really curious about about doing what what I mentioned ear about scaling it up so for example what's the what's the how much can we push this kind of algorithm so a I think at some point if we're going to have enough compute and maybe a faster it is this is like some fast training but probably like if it's implemented in a in parallelizable library or things like that is a you can run larger models you can run a huge number of hyper parameter search and and you can see at some point where it scales up because it's also true like we didn't do I mean yeah there is a like we didn't do a huge hyper parameter search at the end like we did some but it's not but like definitely not comparable to what standard in a machine learning paper in which they try in which they try like thousands of combination of hyper parameters no we did not do grid sweeps of like hyper par yeah yeah exactly so so I think there's there's room for pushing these for sure a little bit more and May hopefully uh much more but this is a I think scalability is something that is really hard to to intuitively predict like why models work well on seven layers and then 10 they collapse is a you just just do it and notice that and then I guess you can do some interpretability afterwards and understand the reason but doing it before is always a a tricky thing and so I guess trying it is probably the yeah if you have a method that allows you to try fast and fail fast is the best way of scaling it up I guess yeah so I would also say you know based on some of the not so publishable results that I've seen in for instance you know trying to replicate hous work with like bouncing mnus digits you know how well this works often does seem to depend on exactly how much structure the generative model provides you you know to give like prediction errors that point in the right Direction

rather than well in exactly the right direction rather than just vaguely the right direction right so you know what is sort of how smooth versus how peaked tightly peaked is the landscape of your like is your free energy landscape and when it's more tightly peaked this algorithm seems to work you know significantly better versus when it's not so tightly peaked it'll often sort of get into the region of valid Solutions and then get very noisy as it becomes like unable to decide which of the potentially valid Solutions is really the best one to converge on you know I would say it's not so good yet at dealing with like multimodal posterior distributions once a good generative model has been learned so they're scaling it up to that you know I'm also interested to see like how far could we take this on a fairly standard machine learning Problem by just saying you know every time we have more layers than would be biologically plausible to differentiate just insert a random variable like parameterize a gaussian right there and then just say okay now you know the amount of gradient calculation you have to do between layers is biologically plausible and will use the predictive coding math to actually you know effectively solve like the we'll have the random variables substitute for back propagation in a certain way what do you mean by that so if you have like a you know 12 layer I think Tomaso you said like a 12 um energy based Transformer or something so I think I think the general concept about biologically biological plausibility is that is a you have a latent variable another latent variable and you want any want the basically Computing the gradients of the parameters to depend on both lat variables and not on other stuff that are in between so for example if you consider the function that goes from one lat variable to the to the first from the first to the second to be a three- Network the gradients of the weights of the intermediate layer or of the first they would depend on the last lat variable and this would Mak basically make you break the locality assumption and biologically biological plausibility while if you consider like a function to be the function to be a single fit forward or convolutional layers it means that for every weight you have a latent variable before it so you have a prec neuron and a postoptic neuron actually in practice and this will basically allow allow you to once you do the once you compute the gradients of the free energy to have everything that depends on PR synaptic and post synaptic neurons so this is the concept with biological plausibility the problem that for example I I'm noticing in in when you do classification is that doing this allows you to propagate the error a little bit less regular L than backprop does because backprop is sequential so it shines a little bit for very deep models especially if you have the right activation function so is a so I guess that's what Eli was mentioning when he was thinking about yeah so if you have for instance 12 if you have for instance image 12 layers classification right then roughly

speaking you know the getting a gradient for the activations of the middle layer of the neural network depends on all the way on the final observation now what we'd like to say is well how much you know ultimately like you know it's been proved that predictive coding in the limit implements backprop just very slowly but what we kind of like to do is to calculate gradients a bit more quickly and so what I was saying is that you know what you could do is sort of take the middle layer and just say well this is going to be a random variable now like instead of you know a single deterministic activation that has to be exactly right in order to pass gradients through this is now going to be a gaussian distribution over activations and having done that you can then use you know our predictive coding algorithm to say I'll try and you know propagate through the latent space of possible you know weights or actually no sorry I'll try to like layer activations find you know a relatively good solution or rather a you know particle cloud of relatively good solutions for those and then update the weights in order to make you know those activations more likely without depending on the final classification so depending on the entire like backwards computation tape backprop very but at the cost of being a little bit random let's say what's wrong with that well nothing say probabilistic machine Learners and say free energy principle people and you know active inference people but um you know currently currently the machine learning world doesn't like it very much they say Bas doesn't scale we're going to teach them they're wrong about that well it's super cool um if if you have any other things you want to add otherwise good luck with the conference and with seeing it's super exciting and I hope people watching this go to the repo continue on some of the explorations that you've mentioned and and explore it in their own ways too yeah I mean feel free to send questions to our email addresses that are listed on the paper if you are not able to get the repo to work or if you make the repo work and you have ideas for follow-ups yes that too yeah we've got a few ideas already like you know I think I was talking to Tomaso and saying like I've been meaning to write some code that will allow you to sort of pass a deterministic output between random variables so that we would you know not just be able to do like graphical models but proper stochastic computation graphs divide and contribute exactly yes cool thank you fellows till next time very much Daniel for organizing this yeah yeah thanks for running these like every time a new one comes out it goes on my watch later list which admittedly I never work through but if they do go on it and sometimes sometimes one or two videos cool cool when I get the opportunity to binge you are what I'm binging thank you see you guys bye have a nice day goodbye